CENTRO DE ENSEÑANZA TÉCNICA Y SUPERIOR



LENGUAJES DE PROGRAMACIÓN ORIENTADA A OBJETOS
Profra. Vanessa Miranda

ProyectoFinal_ICC202_2026
Minuta 3

Presentado por:

Omar Muñoz - 36471
Paul Alvarez - 40683
Karim Herrera - 39219
Abel Martinez - 38531

Tijuana, B.C. a 13 de mayo de 2026

1. Resumen

En este tercer avance se implementaron los sistemas de pisos y puertas (Door y Floor) que estructuran la progresión del juego, el sistema de sinergias entre reliquias, golpes críticos en el motor de combate, la generación de pociones como recompensa post-combate, y las listas estáticas de reliquias y pociones disponibles (RelicPool y PotionPool). La interfaz gráfica SFML continuó avanzando con fondos de pantalla y la estructura base de menús. El proyecto cuenta ahora con una progresion jugable de pisos con escalado de dificultad y un sistema de sinergias funcional.

2. Objetivos Cumplidos

2.1 Sistema de Pisos y Puertas (Door / Floor)

Se diseñó e implementó el esqueleto completo de las clases Door y Floor, que forman el núcleo de la progresión del juego:

- Door encapsula tres tipos de puerta mediante el enum DoorType { NORMAL, CORRUPT, EVENT }. Cada puerta almacena su número (1–3), su tipo y el EnemyType del enemigo que contiene. La corrupción se oculta al jugador intencionalmente: getDisplayName() nunca revela si una puerta es de tipo EVENT.
- Floor genera las tres puertas de cada piso mediante generateDoors(const Player& player), leyendo el modificador corruptDoorModifier del jugador para ajustar la probabilidad de puerta corrupta. Las estadísticas de los enemigos escalan con floorNumber a través de scaleEnemyStats().
- spawnEnemyFromDoor() instancia un Enemy normal o un MiniBoss dependiendo de si la puerta es corrupta.
- giveRewards() entrega reliquias y pociones al jugador al finalizar un combate, consultando al RelicPool según la rareza esperada.
- triggerEvent() maneja las puertas de tipo EVENT (en desarrollo); por ahora otorga beneficios al jugador sin combate.

El código fue desarrollado principalmente por Karim; aunque no se hizo push el día de la reunión, fue integrado al repositorio poco después de manera coordinada con Omar.

2.2 Sinergias de Reliquias

Karim implementó el sistema completo de sinergias entre reliquias en Player:

- checkRelicSynergies(): recorre el inventario del jugador y verifica combinaciones predefinidas de dos reliquias. Al detectar una combinación activa, registra el nombre de la sinergia en el vector activeSynergies.

- hasRelic(const string& relicName): verifica si el jugador posee una reliquia por nombre.
- hasSynergy(const string& synergyName): verifica si una sinergia está activa.

El diseño permite agregar nuevas combinaciones fácilmente extendiendo la lógica dentro de checkRelicSynergies() sin modificar las clases de reliquia individuales.

2.3 Golpes Críticos

Se agregó la mecánica de golpes críticos al CombatEngine. Cuando un ataque físico o especial pasa el check de precisión, existe una probabilidad adicional de que el golpe sea crítico, multiplicando el daño final. El log de combate en la interfaz SFML refleja visualmente cuándo ocurre un crítico.

2.4 RelicPool y PotionPool

Se crearon las listas estáticas de todos los objetos disponibles en el juego:

- RelicPool expone métodos estáticos getRandomRelic(), getRandomCommonRelic(), getRandomRareRelic() y getRandomLegendaryRelic(), permitiendo a Floor entregar reliquias de la rareza correcta según el tipo de encuentro completado.
- PotionPool centraliza las pociones disponibles para el sistema de recompensas.

2.5 Generación de Pociones como Recompensa

Abel integró la generación de pociones al flujo de recompensas de Floor. Al terminar un combate, dependiendo del tipo de enemigo derrotado, el jugador puede recibir una poción en su inventario (sujeto a disponibilidad de slots). Esto conecta el sistema de pociones implementado en el Avance 2 con el flujo real del juego.

2.6 Interfaz SFML (Progreso hacia 50%)

Paul continuó el desarrollo de la interfaz gráfica, trabajando en la integración de fondos de pantalla (assets Corridor.jpeg, CTHomeScreen1.jpg) y en la arquitectura base de los menús que darán soporte a la pantalla de selección de puertas y al flujo completo del juego.

3. Desafíos Enfrentados y Soluciones

3.1 Probabilidad de Corrupción Modificable por Reliquias

Desafío: La probabilidad de puerta corrupta debía escalar con el número de piso base y al mismo tiempo ser ajustable por las ProbabilityRelics que el jugador hubiera acumulado, sin acoplar Floor directamente a las subclases de reliquia.

Solución: Floor::calculateCorruptionChance(float modifier) recibe el valor corruptDoorModifier leído del Player como parámetro y lo suma a la probabilidad base del piso. De esta forma Floor solo conoce al Player por interfaz pública, manteniendo el desacoplamiento.

3.2 Escalado de Estadísticas sin Romper el Balanceo

Desafío: Los enemigos de pisos superiores debían ser notablemente más desafiantes, pero un escalado lineal simple hacía los últimos pisos imposibles o los primeros triviales.

Solución: scaleEnemyStats() dentro de Floor aplica multiplicadores progresivos ajustados mediante pruebas repetidas de combate, dando una curva de dificultad que el equipo consideró satisfactoria durante esta reunión.

3.3 Coordinación del Push de Door y Floor

Desafío: El código de Door y Floor estaba siendo trabajado en la máquina de Karim pero no se integró al repositorio el día de la reunión, generando dependencias bloqueadas para el resto del equipo.

Solución: Se coordinó con Omar para sincronizar el push lo antes posible; el resto del equipo trabajó en paralelo en sus módulos independientes (SFML, pociones) mientras se resolvía la integración.

4. Distribución del Trabajo

Omar: Implementación de Door y Floor, balanceo de estadísticas de personajes y enemigos, implementación de golpes críticos en CombatEngine, corrección de bugs, coordinación del repositorio Git y resolución de conflictos.

Paul: Desarrollo de la interfaz SFML, integración de fondos de pantalla y avance en la arquitectura de menús.

Karim: Diseño e implementación de RelicPool (push posterior a la reunión); sistema de sinergias de reliquias (checkRelicSynergies, activeSynergies, hasRelic, hasSynergy).

Abel: Implementación de la generación de pociones como recompensa post-combate, integrada al flujo de Floor::giveRewards().

5. Estado Actual y Próximos Pasos

5.1 Funcionalidades Completadas

- Sistema de pisos y puertas (Door, Floor) con escalado de dificultad
- Puertas corruptas con probabilidad creciente modificable por reliquias

- Sistema de recompensas post-combate (reliquias y pociones)
- Sinergias de reliquias con detección dinámica
- Golpes críticos en el motor de combate
- RelicPool y PotionPool como fuentes de drops
- Todos los sistemas del Avance 2 consolidados y funcionales

5.2 Pendientes para el Siguiente Avance

- Boss final con sistema de tres fases dinámicas
- RelicTracker: compendio de reliquias desbloqueadas persistente entre sesiones
- Interfaz SFML al 90%: pantalla de selección de personaje, selección de puertas con preview, overlays de Level Up y Move Select
- Clase que juntara la funcionalidad de floor y door con el resto del sistema

6. Conclusión

El tercer avance consolida la progresión estructural del juego: con Door y Floor implementados, el jugador ya tiene el esqueleto para navegar los cinco pisos con dificultad escalada y recibir recompensas reales al finalizar cada encuentro. Las sinergias de reliquias añaden la capa estratégica que diferencia cada partida, y los golpes críticos aportan varianza al combate. El equipo enfrenta los últimos pasos de desarrollo: boss final, interfaz completa e integración total del flujo de juego, con una base sólida sobre la que construir.

7. Minuta de Reunión

Fecha: 13 de mayo de 2026

Modalidad: Presencial

Participantes: Omar Muñoz, Paul Alvarez, Karim Herrera, Abel Martinez



Temas y Acuerdos

- Diseño e implementación de Door (DoorType: NORMAL, CORRUPT, EVENT) y Floor (generación de puertas, escalado de stats, spawn de enemigos, entrega de recompensas)
- Sistema de sinergias de reliquias: checkRelicSynergies(), activeSynergies, hasRelic(), hasSynergy() en Player
- Generación de pociones como recompensa post-combate integrada en Floor::giveRewards()
- RelicPool y PotionPool: listas estáticas de objetos disponibles para drops
- Golpes críticos implementados en CombatEngine
- Balanceo de estadísticas de personajes y enemigos por piso
- Corrección de bugs en el motor de combate y el flujo de puertas
- Avance en la interfaz SFML: fondos de pantalla y estructura de menús